



CC/CBPROG3 Machine Project Specifications

3rd term, AY 2025-2026

Media Vault

A Personal Media Collection Tracker

I. Introduction

We are surrounded by more media than any generation in history — movies, series, books, music albums, podcasts, video games, manga, and more. Yet most people have no reliable system for remembering what they have seen, read, or played; how they felt about it; or what they want to experience next. MediaVault is a personal media collection tracker that solves this problem.

As software developers, your task is to design and build MediaVault: an application that lets a user catalog, organize, and reflect on the media they consume. The system must record what media the user has completed, what they are currently enjoying, and what they plan to experience next. It must also let users attach a personal rating and short review to anything they have finished.

Unlike a generic assignment, MediaVault is personal by design. Each group must build a system around the media types that they actually consume. There is no single correct set of classes — only classes that are well chosen, well justified, and well implemented.

II. The Challenge

You and your partner are junior developers at a small software start-up. Your first client is yourselves: you need a personal media management tool that fits your real habits. You will go through the full software development cycle — requirements analysis, object-oriented design, prototyping, and final delivery — across two phases.

The project has two major phases:

- Phase 1 (MCO1): An object-based, text-driven prototype of MediaVault, demonstrating correct class design and core functionality without a graphical interface.
- Phase 2 (MCO2): A fully object-oriented, GUI-based implementation of MediaVault following the Model-View-Controller (MVC) design pattern.

III. Media Type Selection

Each group must select at least three (3) distinct media types on which to base their system. These media types will define your concrete subclasses and much of your design. Choose media types that genuinely reflect your group's interests — you will be asked to justify these choices in your Design Rationale, and you will discuss them during your demo.

⚠ Important: Personalization is Graded

Your media type selection must be specific and personal. Vague choices such as “entertainment” or “content” are not acceptable.

If two groups submit the same set of three media types with the same class structure, this will be flagged for review under academic honesty policies.

Your design rationale (see Section V) must explain why your group chose each media type, connecting it to your actual interests and habits.

Suggested media types (choose at least 3, or propose your own with instructor approval):

- Movies
- TV Series (local or international)
- Anime
- Books (fiction or non-fiction)
- Manga, Manhwa, or Webtoons
- Music Albums or Singles
- Podcasts
- Video Games
- Board Games or Card Games
- Stage Productions (theatre, concerts, etc.)

Note: At least one of your chosen media types must be episodic in nature (i.e., it has multiple episodes, volumes, or installments — such as a Series, Anime, Manga, or Podcast), and at least one must be standalone (i.e., experienced as a single unit — such as a Movie, Book, or Music Album). This distinction is important because it will drive a key design decision discussed in Section V.

IV. Minimum Requirements

A. MCO1 — Object-Based Prototype

The Phase 1 prototype must demonstrate correct object-based design and implementation. At minimum, the following must be present:

Classes and Structure

- At least three (3) classes for types media to be selected by the group (see Section III).
- At least one (1) of these classes must represent an episodic media type (e.g., Series, Anime, Manga).
- At least one (1) of the classes must represent a standalone media type (e.g., Movie, Book, Album).
- A Library or Collection class that manages a list of MediaEntry objects. This class must support adding, removing, and retrieving entries.
- A User or Profile class that represents the person using the system, linked to their Library.

Status and Rating — The Design Constraint



Required Design Constraint: Status-Gated Ratings

Every media entry must have a status. The three valid statuses are:

- Planned — the user intends to consume this media in the future.
- In Progress — the user is currently consuming this media.
- Completed — the user has finished this media.

A personal rating (e.g., a score from 1 to 10) and an optional short review may **ONLY** be attached to an entry with a Completed status. Your design must enforce this rule.

How you enforce this rule is a design decision that is entirely yours to make. You may use a separate class (e.g., CompletedEntry), a status-checking method, a specific inheritance structure, or another approach altogether. There is no single correct answer. However, you must justify your chosen approach in your Design Rationale (see Section V), and you must also describe at least one alternative you considered and explain why you did not choose it.

You will be assessed not just on whether the rule works, but on the quality of your reasoning.

Functionality

- Add a new media entry to the library (with a chosen status of Planned or In Progress upon addition).
- Update the status of an existing entry (e.g., mark an In Progress entry as Completed).
- Assign a rating and review to a Completed entry; the system must prevent ratings on non-Completed entries.
- Display all entries in the library, with the option to filter by status or media type.
- Display a summary of the library (e.g., total entries, count per status, average rating of Completed entries).
- Text-based menu-driven interface using the console (no GUI for MCO1).

B. MCO2 — Object-Oriented, GUI-Based Application

The Phase 2 application must extend the prototype into a fully object-oriented, GUI-based system. At minimum, the following must be present:

- All requirements from MCO1 must be met, refined, and extended. In addition, include:
 - An abstract base class `MediaEntry` (or a name of your choice) that represents any item in the library. It must contain attributes and methods common to all media types.
 - At least three (3) concrete subclasses of `MediaEntry`, corresponding to the media types selected by the group (see Section III).
 - At least one (1) of the subclasses must represent an episodic media type (e.g., Series, Anime, Manga) with attributes specific to serialized content (such as episode count, chapter count, current episode/chapter, etc.).
- Implementation must use the Model-View-Controller (MVC) design pattern, clearly separating Model classes (`MediaEntry`, `Library`, `User`, etc.), View classes (GUI panels, windows), and Controller classes (event handlers, business logic coordinators).
- A graphical user interface built using Java Swing or JavaFX.
- File I/O: the library must be saveable and loadable between sessions (text file format, not a database).
- At least one additional feature beyond the MCO1 requirements, proposed by the group (e.g., a search function, a tag/genre system, a statistics dashboard, or a recommended media list based on ratings).
- Inheritance must be used meaningfully (e.g., at least one abstract method overridden differently across subclasses).
- Exception handling must be applied for file I/O and invalid user input.

V. Design Rationale Document

Each phase submission must include a Design Rationale document submitted alongside the UML class diagram. This document is a written explanation of the key decisions made during the design of your system. It is not a description of your code — it is a justification of your design.

The Design Rationale is a core deliverable, not an optional reflection. Submissions without a Design Rationale, or with a rationale that does not address the required sections, will incur deductions in the Designing Classes/Objects criterion of the rubric.

The Design Rationale must address all of the following:

1. **Media Type Selection:** Explain why each of your chosen media types was selected. Connect each choice to your group's actual interests or habits. Describe at least one attribute or behavior that is unique to each of your concrete subclasses and would not belong in the abstract base class.
2. **[For MCO2 only] Abstract Class Design:** Explain which attributes and methods belong in your abstract base class, and why. What did you decide to leave out of the abstract class, and why? How does your hierarchy reflect the principle of information hiding?
3. **The Rating Constraint:** Describe how your design enforces the rule that ratings may only be attached to Completed entries. Describe at least one alternative approach you considered (e.g., if you used a separate class, describe what a method-only approach would have looked like, and why you preferred your chosen solution). There is no right or wrong answer here — only well-reasoned or poorly-reasoned ones.
4. **One Additional Decision:** Identify one other design decision in your system that was not straightforward — a moment where your group had to choose between two or more valid approaches. Describe both options and explain why you chose the one you did.

The Design Rationale should be approximately 1–2 pages in length. It need not be exhaustive — quality of reasoning matters far more than length.

VI. Version Control Requirements

The use of version control via Git is a **required** component of this project, not an optional one. Both members of the group are expected to maintain and contribute to a shared Git repository throughout the development of both phases.

Git Repository Requirements

Both group members must have commits in the repository. A repository where only one member has commits will result in deductions for the non-contributing member.

Commits must be small and meaningful — a single commit containing the entire project submitted the night before the deadline will not be credited as good version control practice. Commit messages must be descriptive. Messages such as “final”, “update”, or “fix” are not acceptable. A good message describes what changed and why (e.g., “Add rating validation logic to CompletedEntry class”).

The repository must be private and shared only with your group members. Do not make the repository public until after the grading period.

A link to the repository must be included in your submission.

Commit history must span the entire development period, not just the final days before the deadline.

Recommended platforms: GitHub, GitLab, or Bitbucket. If you are new to Git, you may use GitHub Desktop or the Git integration in your IDE of choice. All CCPROG3 students are expected to be able to compile and run their programs from the command line — the same applies to cloning and basic Git operations. Online platforms are available to help you learn Git. For example, <https://gitmastery.me/>

Version control practice is assessed as part of MCO2 bonus points (see Section X). However, even for MCO1, the presence of a well-maintained repository with commits from both members will be taken into account during the demo as evidence of genuine collaboration and incremental progress.

VII. Milestones

Milestone	Description	Deadline
Media Type Proposal	A short (half-page) document naming your group’s chosen media types and a one-sentence justification for each. No class diagram required yet. Submit via AnimoSpace.	Week 4 — before class
MCO1 Submission	Object-based prototype (see Section VIII for full deliverable list). Includes UML diagram, Design Rationale, source code, test script, and video demo.	11 Jul 2026 (Sat) before 9:00 PM
MCO2 Submission	GUI-based, MVC-structured application (see Section VIII). Includes updated UML diagram, updated Design Rationale, source code, test script.	3 Aug 2026 (Mon) before 7:30 AM
MCO2 Demo	Live individual demo. Schedule to be announced. Details in Section XII.	3 Aug – 7 Aug 2026

VIII. Deliverables

The deliverables for both MCO1 and MCO2 are as follows. Unless otherwise noted, all items apply to both phases.

- **Signed Declaration of Original Work** — See Appendix A for the template. Submit as a flattened PDF. The declaration must explicitly state that no generative AI tools were used in the design or implementation of the project.
- **UML Class Diagram** — Submitted as a PDF or PNG. Must follow correct UML notation. Must be legible and well-organized. Diagrams that cannot be read due to size or clutter will not be credited.
- **Design Rationale Document** — As described in Section V. Submitted as a PDF. Must address all four required sections.
- **Source Code** (ZIP) — All .java files with internal documentation. Must compile and run from the command line using the standard JDK. Include any required external libraries.
- **Git Repository Link** — A link to your private repository. The repository must be accessible by your instructor at the time of submission (add your instructor as a collaborator if the repository is private).
- **Javadoc-generated External Documentation** — For all proponent-defined classes. Must include pre-conditions, post-conditions, parameters, and return values for all non-trivial methods.
- **Test Script** — Following the format in Appendix B. At least 3 unique test cases per non-trivial method across all classes. Getter and setter methods are exempt. Methods that only print to the screen with no logic are also exempt.
- **Video Demo** (MCO1 only) — All members must appear and contribute equally. The demo must show the program running and explain key aspects of the design. A demo script will be provided closer to the due date.

All deliverables are to be bundled and submitted via AnimoSpace. Late submissions will not be accepted.

IX. Submission

All deliverables for both MCO1 and MCO2 must be submitted via AnimoSpace. Submissions through any other channel (email, messenger, USB drive, etc.) will not be accepted. Please observe all deadlines as published on AnimoSpace. Late submissions are automatically graded as zero.

X. Bonus Points

Bonus points apply to MCO2 only, and only if the submission meets all minimum requirements (Exemplary rating for Program Correctness and Designing Classes/Objects). Up to 10 bonus points may be awarded in total.

Possible sources of bonus points include (but are not limited to):

- Implementation of an advanced additional feature (e.g., a social sharing export, import from a public API such as the Open Library API or IGDB, or a recommendation engine based on the user's rating history). Points awarded based on complexity.
- A statistics or analytics dashboard within the GUI that visualizes the user's media habits (e.g., ratings distribution chart, media type breakdown, monthly activity).

Version Control Bonus (up to 4 points)

Up to 4 bonus points may be awarded for effective use of version control throughout the project. Awarded based on:

- Frequency and distribution of commits (small, regular commits over time — not a single large commit near the deadline).
- Quality of commit messages (descriptive, specific, and informative).
- Evidence of contribution from both group members across the commit history.
- Use of branches (e.g., a feature branch for the GUI, a separate branch for file I/O, merged back into main).

Note: Bonus points from all sources are capped at 10. Do not spend effort on bonus features until all minimum requirements are fully and correctly met.

XI. Grading

For grading criteria and rubrics, please refer to Appendices A and B of the course syllabus. Note that the Design Rationale document is evaluated within the “Designing Classes/Objects” criterion. A submission that lacks a Design Rationale, or whose rationale does not address the required sections, will not receive an Exemplary rating for that criterion regardless of how well the UML diagram is drawn.

XII. Demo

MCO1 Demo — Video Submission

The MCO1 demo is submitted as a video. All group members must appear in the video and must have a roughly equal share of the presentation. The video must demonstrate the program running from the command line and must briefly walk through the key design decisions in the submitted class diagram. A demo script will be provided by the faculty closer to the submission date to help groups cover the expected content. Keep the video concise; do not include material unrelated to the project.

Any member who does not appear in the video will receive a grade of zero for MCO1.

MCO2 Demo — Live Individual Demo

The MCO2 demo is conducted live during class time (or at an additional schedule announced by the faculty). The demo has two parts:

- **Group Demo:** Both members demonstrate the running application together, guided by a faculty-provided demo script. All features must be demonstrable from the compiled program.
- **Individual Demo Problem:** Each student is given a short individual programming task related to their own submitted project — for example, adding a new method to one of their own classes, or extending a feature they already built. The task is designed to test whether each student genuinely understands their own codebase. Students are assessed individually for this portion.

Do not be absent or late on announced demo days. A group or individual who is absent during the demo will receive a grade of zero for that phase. A program that does not compile and run during the demo will also receive a grade of zero.

What to Expect During the Demo

Faculty may ask any group member to explain any part of the submitted design or code. Faculty may ask “why did you not do it this way?” questions — presenting an alternative design and asking for a comparison.

Faculty may ask a member to trace through a specific method or explain what would break if a particular design decision were changed.

Students who cannot explain their own design and code convincingly will receive a grade of zero for the demo component, regardless of the quality of the submitted code.

XIII. Collaboration and Academic Honesty

This project is to be developed in pairs for both phases. In exceptional cases (approved by the instructor), a student may work alone. Groups of more than two members are not permitted under any circumstances.

A student may not discuss the design or implementation of the project with anyone other than their group partner and the instructor. Questions about the project specifications should be raised in the AnimoSpace Discussion page so that all students benefit from the clarification. Copying another group's design, code, or documents — or allowing your work to be copied — is a violation of academic honesty policies and will result in a grade of 0.0 for the entire course, in addition to a case filed with the Discipline Office.

On the use of Generative AI: as stated in the course syllabus, the use of generative AI tools (such as ChatGPT, GitHub Copilot, Claude, Gemini, or any similar tool) for any graded deliverable in this course is strictly prohibited. This includes using AI to generate, complete, or significantly revise any part of your class diagram, Design Rationale, source code, test script, or documentation. Violations will be treated with the same severity as plagiarism.

Each submission must be accompanied by a signed Declaration of Original Work (see Appendix A). By signing the declaration, both group members affirm that no generative AI tools were used in any part of the submitted work.

XIV. Documentation and Coding Standards

All source code must include internal documentation (comments). At a minimum, every class and every non-trivial method must have a Javadoc comment with a description, `@param`, `@return`, and — where applicable — `@pre` and `@post` conditions. You may use an IDE or the command-line javadoc tool to generate external documentation from these comments.

Inline comments should be used for complex logic blocks. You are not expected to comment every line, but a reader unfamiliar with your code should be able to understand what any given block does by reading the surrounding comments and identifiers.

Coding standards from the course (naming conventions, indentation, etc.) must be followed. A well-named variable often eliminates the need for an inline comment; poor naming requires more explanation. Both contribute to the readability score in the rubric.

XV. Other Notes

- You are required to use classes and methods wherever appropriate. No brute-force solutions.
- The project must be written in Java. The minimum JDK version is 17.
- For MCO1, object-based design (no inheritance) is expected. For MCO2, full object-oriented design with inheritance, polymorphism, and exception handling is required.
- Statements and methods not covered in class may be used, but students are responsible for learning and implementing them independently.
- External libraries are permitted. Include all required .jar files in your ZIP submission and document their usage in the Declaration of Original Work.
- When using any external resource (images, icons, sound files), cite your sources in the Declaration of Original Work using APA format.

Appendix A — Declaration of Original Work

Declaration of Original Work

We/I, [Your Name(s)] of Section [Section], declare that the code, resources, and documents submitted for the [1st / 2nd] phase of the Major Course Output (MCO) for CCPROG3 are entirely our own work and effort. We take full responsibility for this submission and understand the consequences of academic dishonesty as stated in the DLSU Student Handbook.

We further declare that no generative AI tools (including but not limited to ChatGPT, GitHub Copilot, Claude, Gemini, or similar applications) were used in the design, implementation, testing, or documentation of this project.

[If applicable] We acknowledge the following external sources used in this project:

- Author. Year. Title. Publisher. Link.
- Author. Year. Title. Publisher. Link.

By signing below, we affirm the authenticity and originality of our work.

Signature and Date:

[Student 1 Name]
[ID Number]

Signature and Date:

[Student 2 Name]
[ID Number]

Note: Do not submit documents where your digital signature is easily accessible. Ideally submit a flattened PDF. You may use the eSignature feature of Google Docs or equivalent tools.

Appendix B — Test Script Format

The test script must follow the format shown in the example below. Provide at least 3 unique test cases per non-trivial method across all proponent-defined classes.

Class: MediaEntry (example)						
Method	#	Test Description	Sample Input	Expected Output	Actual Output	P/F
setRating()	1	Rating accepted when status is Completed	status=Completed, rating=8	rating stored as 8		
	2	Rating rejected when status is Planned	status=Planned, rating=7	Exception or rating unchanged		
	3	Rating rejected when status is In Progress	status=InProgress, rating=5	Exception or rating unchanged		
updateStatus()	1	Status updated from Planned to In Progress	current=Planned, new=InProgress	Status is InProgress		
	2	Status updated from In Progress to Completed	current=InProgress, new=Completed	Status is Completed		
	3	Invalid status string rejected	current=Planned, new="done"	Exception thrown		

Include a separate table for each class. Getter and setter methods without logic are exempt. Methods that only display output to screen with no computation are also exempt.

Appendix C — Design Rationale Template

Use the following template as a guide for your Design Rationale document. You may reorganize sections as needed, but all four required topics must be addressed. The rationale must be written in your own words. Aim for clarity and conciseness over length.

File: DesignRationale_[GroupName]_MCO[1 or 2].pdf

Submit as a PDF. Recommended length: 1–2 pages.

1. Media Type Selection

Identify each of your chosen media types. For each one:

- Why did your group choose this type? (Connect it to your actual interests.)
- What attribute or behavior does this subclass have that the abstract base class does not?
- Why does that attribute or behavior not belong in the base class?

2. Abstract Class Design

Describe what you placed in your abstract base class and why. Address the following:

- What attributes are common to all media types and therefore belong in the base class?
- What methods did you define in the base class, and which (if any) are abstract? Why?
- What did you deliberately leave out of the base class, and where did it go instead?

3. The Rating Constraint

Describe how your design enforces the rule that ratings may only be given to Completed entries. Address the following:

- What specific design approach did you use (e.g., a separate subclass, a guard in a method, a status check, etc.)?
- Describe at least one alternative approach you considered. What would it have looked like?
- Why did you prefer your chosen approach over the alternative?

4. One Additional Design Decision

Identify one other design decision in your system that was not immediately obvious — a point where your group had to choose between two or more valid options. Describe:

- What were the options?
- What did you choose?
- Why?